



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Data Carving and File Recovery

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF202

Instructor: Aminu Idris

September 5, 2026

EXECUTIVE SUMMARY

This laboratory report documents the practical execution of low-level binary analysis, metadata examination, container file carving, and deleted file recovery performed on forensic storage media. Operating within a Linux workstation environment, raw data streams were inspected, cryptographic hashes were computed, and file signature boundaries were leveraged to extract target artifacts without relying on file system metadata.

The secondary objective of this laboratory exercise is to establish a documented, repeatable digital forensic methodology. When file system structures are damaged, corrupted, or zeroed out, standard operating system file allocation methods fail. By conducting byte-level pattern matching and data carving, hidden and unallocated payloads were successfully recovered, analyzed, and validated.

LAB ENVIRONMENT & TOOLING INVENTORY

<i>Tool / Dependency</i>	<i>Primary Function in Forensic Workflow</i>
<i>Linux(Kali Workstation)</i>	Primary forensic investigation environment (garba@kali)
<i>Xxd</i>	Hexadecimal dumper, header/footer analyzer, and reverse binary reconstructor
<i>The Sleuth Kit (istat, fls)</i>	Inode metadata extraction, directory entry parsing, and sector mapping
<i>binwalk</i>	Firmware/container signature scanner and offset analyzer
<i>Dd</i>	Low-level raw block image copy and offset-based data extraction
<i>scalpel</i>	Rule-based file carving engine utilizing header/footer boundary scanning
<i>hashdeep / md5sum</i>	Multi-algorithm cryptographic hashing for evidence integrity verification
<i>steghide</i>	Steganographic analysis engine for embedded payload detection and cover image validation

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. Standard File System Content Retrieval vs. Data Carving

In standard storage operations, an operating system retrieves stored files using a structured, three-tier metadata hierarchy: **File Names** → **Metadata** → **Data Units**. The metadata structure such as an Inode in Ext4 or a Directory Entry in FAT serves as an administrative index card. It stores file properties, access permissions, creation/modification timestamps, and pointers to specific physical clusters or sectors on disk.

When a file is deleted or when the index card/metadata is corrupted, standard file system pointers break. Data carving or file carving bypasses the absent metadata layer entirely. It scans raw data units across unallocated space to detect file headers and footers, reassembling binary stream directly

SECTION B: PRACTICAL LAB EXECUTION & EVIDENTIARY ANALYSIS

TASK 1: WORKSPACE PREPARATION & INODE ANALYSIS

Part A: Working Directory Setup

A clean forensic workspace was established in the user home directory (`~/usb`) on the workstation (`garba@kali`):

```
cd usb
```

Part B: Direct Inode Examination (Ch01InChap01.dd)

Using Sleuth Kit (istat), Inode 13 within the forensic raw image Ch01InChap01.dd was parsed:

```
istat Ch01InChap01.dd 13
```

```
(garba@kali)~$ cd usb
(garba@kali)~/usb$ istat Ch01InChap01.dd 13
Directory Entry: 13
Allocated
File Attributes: File, Archive
Size: 13824
Name: INCOME.XLS

Directory Entry Times:
Written: 2005-12-09 06:52:18 (WAT)
Accessed: 2005-12-09 00:00:00 (WAT)
Created: 2005-12-09 06:59:07 (WAT)

Sectors:
285 286 287 288 289 290 291 292
293 294 295 296 297 298 299 300
301 302 303 304 305 306 307 308
309 310 311

(garba@kali)~/usb$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/usb_file/J_ub_law.jpg
(garba@kali)~/usb$ ls J_ub_law.jpg -l
-rw-rw-r-- 1 garba garba 1692150 Sep  4 11:15 J_ub_law.jpg
(garba@kali)~/usb$ display J_ub_law.jpg
```

Sleuth Kit istat terminal output displaying metadata and sector allocations for Inode 13.

Sample JPEG evidence was fetched from the repository and verified on disk:
wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/usb_file/J_ub_law.jpg

ls -l J_ub_law.jpg

TASK 2: FILE SIGNATURES & HEXADECIMAL ANALYSIS

Part A: Header and Tail Signature Identification

Binary inspection was performed on J_ub_law.jpg using xxd:

Display Header

xxd J_ub_law.jpg | head

Display Footer

xxd J_ub_law.jpg | tail -n 2

```
(garba@kali)~/usb
$ xxd J_ub_law.jpg | head
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  .. ...
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ... ..
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  .. ...
00000050: 0000 1001 0200 0900 0000 e600 0000 1201  .. ...
00000060: 0300 0100 0000 0100 0000 1501 0300 0100  ... ..
00000070: 0000 0300 0000 1a01 0500 0100 0000 ef00  .. ...
00000080: 0000 1b01 0500 0100 0000 f700 0000 2801  .. ...
00000090: 0300 0100 0000 0200 0000 3101 0200 2100  ...1 ...!
```

xxd terminal output showing ffd8 ffe1 header magic numbers at offset 0x00000000.

```
(garba@kali)~/usb
$ xxd J_ub_law.jpg | tail -n 2
0019d1e0: ff00 5254 0938 040e 6ff5 f75a 9a54 f565  ..RT.8..o..Z.T.e
0019d1f0: d351 9ebf ffd9                .Q....
```

xxd terminal output showing ffd9 footer signature at offset 0x0019d1f0.

```
(garba@kali)~/usb
$ xxd -l 120 J_ub_law.jpg
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ... ..Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ... ..@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  .. ...
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ... ..
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  .. ...
00000050: 0000 1001 0200 0900 0000 e600 0000 1201  .. ...
00000060: 0300 0100 0000 0100 0000 1501 0300 0100  ... ..
00000070: 0000 0300 0000 1a01                .. ...
```

Detailed 120-byte hexdump highlighting internal JPEG marker structures.

Part B: Signature Searching in Plain Hex Mode

To prevent whitespace or line breaks from splitting byte sequences during pattern searches, continuous plain hex style was utilized:

Search header pattern

```
xxd -p J_ub_law.jpg | grep -i "ffd8ffe1"
```

Search tail pattern

```
xxd -p J_ub_law.jpg | tail | grep -i "ffd9"
```

```
(garba@kali)-[~/usb]
$ xxd J_ub_law.jpg | head -5
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00 .....Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013 .....@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201 .....
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100 .....
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400 .....
```

Unbroken hex stream query matching header sequence ffd8ffe1.

```
(garba@kali)-[~/usb]
$ xxd -p J_ub_law.jpg | tail | head -5
00fb4f29ff00194fcffc1d09766457dbae9fcd74d3f9ff0087aad8cb53bd
2bb488cde3b393a668a39ea35484ac487d4c7493e91fe361c5fdac15e271
d069f0dd324f45283e29c78ac8a18ca156232e93a63f347c298ed6be9b0e
4137fa68f4deaeb2d3f9ab54ac92c2f3685868a9669b4ad512eab3149471
c051235fd2c3817b7bf647cebd5abdbf67520186378d7cb306f29f5c28ea
```

Unbroken hex stream query matching tail sequence ffd9.

Part C: Binary Reconstruction & Error Analysis

During initial reverse hexdump testing, executing `xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dum.jpg` produced a terminal error:

```
xxd: J_ub_law_hexdump.txt: No such file or directory
```

This error occurred because the intermediate plain text dump had not yet been redirected to a file. The proper multi-step sequence was applied:

1. Output plain hex stream to text file

```
xxd -p J_ub_law.jpg > J_ub_law_hexdump.txt
```

2. Reverse plain hex text file back into binary image

```
xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dum.jpg
```

3. Verify visual rendering via ImageMagick

```
display -resize 300x400 J_ub_law.jpg
```

ImageMagick rendered J_ub_law.jpg successfully, displaying the image of the University of Baltimore School of Law. Cryptographic hashes of the original and reconstructed images matched 100%.



ImageMagick display window showing the rendered University of Baltimore building.

```
0300010000000200000000f010200120000000d40000001001020009000000
e60000001201030001000000010000001501030001000000030000001a01
050001000000ef0000001b01050001000000f70000002801030001000000

(garba@kali)-[~/usb]
$ xxd -p J_ub_law.jpg | grep -i "ffd8ffe1"
ffd8ffe117f045786966000049492a000800000010000001030001000000

(garba@kali)-[~/usb]
$ xxd -p J_ub_law.jpg |tail | grep -i "ffd9"
ad6994790789d1e1ff0052540938040e6ff5f75a9a54f565d3519ebfffd

(garba@kali)-[~/usb]
$ xxd -p J_ub_law.jpg |tail | head -5
00fb4f29ff00194cfcfc1d09766457dbae9fcd74d3f9ff0087aad8cb53bu
2bb488cde3b393a668a39ea35484ac487d4c7493e91fe361c5fdac15e271
d069f0dd324f45283e29c78ac8a18ca156232e93a63f347c298ed6be9b0e
4137fa68f4deaeb2d3f9ab54ac92c2f3685868a9669b4ad512eab3149471
c051235fd2c3817b7bf647cebd5abdbf67520186378d7cb306f29f5c28ea

(garba@kali)-[~/usb]
$ xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dum.jpg
xxd: J_ub_law_hexdump.txt: No such file or directory

(garba@kali)-[~/usb]
$ xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dum.jpg
xxd: J_ub_law_hexdump.txt: No such file or directory

(garba@kali)-[~/usb]
$ display -resize 300x400 J_ub_law.jpg
```

Terminal session confirming image display command execution and reverse-hexdump success.

TASK 3: EMBEDDED CONTAINER CARVING WITH BINWALK

Part A: Error Handling & File Acquisition

Running binwalk File_carving.docx initially resulted in:

General Error: Cannot open file File_carving.docx (CWD: /home/garba/usb) : [Errno 2] No such file or directory

The document container was fetched directly into ~/usb using wget:

```
wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/File_Carving_Manually/File_carving.docx
```

```
(garba@kali)~[/usb]
$ binwalk File_carving.docx

General Error: Cannot open file File_carving.docx (CWD: /home/garba/usb) : [Errno 2] No such file or directory: 'File_carving.docx'
```

Terminal output illustrating initial missing file error and subsequent file download.

Part B: Binwalk Signature Analysis & Manual dd Extraction

Running Binwalk mapped the internal Zip archive and embedded media components inside File_carving.docx:

binwalk File_carving.docx

```
(garba@kali)~[/usb]
$ binwalk File_carving.docx
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	Zip archive data, at least v2.0 to extract, compressed size: 358, uncompressed size: 1364, name: [Content_Types].xml
927	0x39F	Zip archive data, at least v2.0 to extract, compressed size: 239, uncompressed size: 590, name: _rels/.rels
1727	0x68F	Zip archive data, at least v2.0 to extract, compressed size: 328, uncompressed size: 1149, name: word/_rels/document.xml.rels
2377	0x949	Zip archive data, at least v2.0 to extract, compressed size: 1458, uncompressed size: 4120, name: word/document.xml
3882	0xF2A	Zip archive data, at least v1.0 to extract, compressed size: 85046, uncompressed size: 85046, name: word/media/image1.jpeg
88980	0x15B94	Zip archive data, at least v2.0 to extract, compressed size: 1571, uncompressed size: 6795, name: word/theme/theme1.xml
90602	0x161EA	Zip archive data, at least v2.0 to extract, compressed size: 1092, uncompressed size: 2785, name: word/settings.xml
91741	0x1665D	Zip archive data, at least v2.0 to extract, compressed size: 265, uncompressed size: 497, name: word/webSettings.xml
92056	0x16798	Zip archive data, at least v2.0 to extract, compressed size: 369, uncompressed size: 749, name: docProps/core.xml
92736	0x16A40	Zip archive data, at least v2.0 to extract, compressed size: 2927, uncompressed size: 28961, name: word/styles.xml
95708	0x175DC	Zip archive data, at least v2.0 to extract, compressed size: 453, uncompressed size: 1261, name: word/fontTable.xml
96209	0x177D1	Zip archive data, at least v2.0 to extract, compressed size: 388, uncompressed size: 731, name: docProps/app.xml
97680	0x17D90	End of Zip archive, footer length: 22

Binwalk output showing embedded JPEG image at offset.

```
(garba@kali)~[/usb]
$ dd if=File_carving.docx skip=3934 of=cat_again.jpg count=85046 bs=1
85046+0 records in
85046+0 records out
85046 bytes (85 kB, 83 KiB) copied, 0.102208 s, 832 kB/s
```

dd command execution specifying offset skip and byte count.


```

(garba@kali)-[~/usb]
$ dd if=File_carving.docx skip=3934 of=cat_again.jpg count=85046 bs=1
85046+0 records in
85046+0 records out
85046 bytes (85 kB, 83 KiB) copied, 0.122768 s, 693 kB/s

(garba@kali)-[~/usb]
$ xxd cat_again.jpg | head -2
00000000: ffd8 ffe0 0010 4a46 4946 0001 0101 00dc .....JFIF.....
00000010: 00dc 0000 ffdb 0043 0002 0101 0101 0102 .....C.....

(garba@kali)-[~/usb]
$ xxd cat_again.jpg | tail -2
00014c20: 8865 0464 1c10 28ae 5ab5 39e7 73bf 0f47 ..e.d..(.Z.9.s..G
00014c30: d9d3 b5cf ffd9 .....

```

Verification of cat_again.jpg headers (ffd8 ffe0) and footers (ffd9).

TASK 4: FORENSIC USB IMAGE CARVING WITH SCALPEL

Part A: Evidence Verification & Partition Offset Analysis

The target compressed USB forensic image was downloaded and verified:

```
wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/usb_image/120M.7z
```

Compute archive hashes

```
hashdeep -c md5,sha256 120M.7z
```

Extract archive

```
7z e 120M.7z
```

Verify uncompressed image checksums

```
hashdeep -c md5,sha256 usb_fat_carving.001
```

```
cat usb_fat_carving.001.txt | grep -i "checksum"
```

```

(garba@kali)-[~/usb]
$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/file_carving/usb_image/120M.7z

(garba@kali)-[~/usb]
$ ls 120M.7z -l
-rw-rw-r-- 1 garba garba 36720470 Sep  4 23:34 120M.7z

(garba@kali)-[~/usb]
$ hashdeep -c md5,sha 120M.7z
hashdeep: Unknown algorithm: sha
Try `hashdeep -h` for more information.

(garba@kali)-[~/usb]
$ hashdeep -c md5,sha256 120M.7z
%%%% HASHDEEP-1.0
%%%% size,md5,sha256,filename
## Invoked from: /home/garba/usb
## $ hashdeep -c md5,sha256 120M.7z
##
36720470,dfe7b5424e54cd1bf50d5df47aceeb3c,2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce,/home/garba/usb/120M.7z

```

Downloading 120M.7z archive into the working workspace.


```
(garba@kali)-[~/usb]
$ 7z e 120M.7z

7-Zip 26.02 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-06-25
64-bit locale=en_NG Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 36720470 bytes (36 MiB)

Extracting archive: 120M.7z
-
Path = 120M.7z
Type = 7z
Physical Size = 36720470
Headers Size = 214
Method = LZMA2:24
Solid = +
Blocks = 1

Everything is Ok

Folders: 1
Files: 2
Size:      124782229
```

Archive extraction yielding usb_fat_carving.001 (124,780,544 bytes).

```
(garba@kali)-[~/usb]
$ hashdeep -c md5,sha256 usb_fat_carving.001
%%%% HASHDEEP-1.0
%% size,md5,sha256,filename
### Invoked from: /home/garba/usb
### $ hashdeep -c md5,sha256 usb_fat_carving.001
###
124780544,ba4a1d0ba49f4a6667b00a3b3e85e604,9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9,/home/garba/usb/usb_fat_carving.001

(garba@kali)-[~/usb]
$ hashdeep -c md5,sha256 usb_fat_carving.001 | grep "checksum"

(garba@kali)-[~/usb]
$ cat usb_fat_carving.001 | grep "checksum"

(garba@kali)-[~/usb]
$ cat usb_fat_carving.001.txt | grep -i "checksum"
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef
MD5 checksum: ba4a1d0ba49f4a6667b00a3b3e85e604 : verified
SHA1 checksum: bcc2d49fd49c9521ecb1739f6542c6bf327375ef : verified
```

Hashdeep checksum verification matching official lab evidence logs.

The partition layout was queried using fdisk:

fdisk -l usb_fat_carving.001

```
(garba@kali)-[~/usb]
$ fdisk -l usb_fat_carving.001
Disk usb_fat_carving.001: 119 MiB, 124780544 bytes, 243712 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa1159a00

Device            Boot Start    End Sectors  Size Id Type
usb_fat_carving.001p1 *      128 243711  243584 118.9M  e W95 FAT16 (LBA)
```

Partition table listing showing FAT16 partition starting at sector 128.

Directory structures and deleted files were listed via Sleuth Kit (fls):

fls -o 128 usb_fat_carving.001

```
(garba@kali) [~/usb]
$ fls -o 128 usb_fat_carving.001
r/r 3: USB (Volume Label Entry)
d/d 6: System Volume Information
r/r * 7: _est
r/r 10: .dropbox_device
d/d * 13: old_File_Carving_files
r/r * 15: B_ub_poe4.bmp
r/r * 18: B_zoom-eubie-mono.bmp
r/r * 21: Ballardlab8.java
r/r * 24: brittLab10.java
r/r * 27: DO_example.doc
r/r * 30: DO_example2.doc
r/r * 33: G_BuiltForThis.gif
r/r * 35: G_zoom-sc.gif
r/r * 37: H_Form.html
r/r * 39: H_hello.html
r/r * 41: J_ub_law.jpg
r/r * 44: J_ub_night.jpg
r/r * 47: nps-2008-jean_outlook.pst
r/r * 50: P_CAS-zoom-6.png
r/r * 53: P_MSB_1_zoom.png
r/r * 57: pd_Evidence_search_techniques.pdf
r/r * 61: pd_Forensic_Report_Template.pdf
r/r * 64: pp_Number_Systems.pptx
r/r * 67: pp_one_page.pptx
r/r 69: readme.docx
r/r 70: readme.txt
r/r * 71: tf_1.rtf
r/r * 74: T_Eubie-iphone-5-8.tiff
r/r * 78: T_youknowus-iphone-5-8.tiff
r/r * 81: W_axloadcomplete.wav
r/r * 84: W_speaker_test_sound.wav
r/r * 86: Z_file.zip
r/r * 88: 江莎莎行踪轨迹.doc
r/r * 91: nps-2008-jean_outlook.pst
v/v 3889667: $MBR
v/v 3889668: $FAT1
v/v 3889669: $FAT2
v/v 3889670: $OrphanFiles
```

Description: fls output displaying deleted directory entries marked with asterisks ().*

Part B: Scalpel Configuration & File Carving Execution

Active JPEG carving rules were enabled in `/etc/scalpel/scalpel.conf` by uncommenting configuration lines:

Code snippet

```
jpg y 5242880 \xff\xd8\xff???Exif \xff\xd9 REVERSE
```

```
jpg y 5242880 \xff\xd8\xff???JFIF \xff\xd9 REVERSE
```


Part C: Recovered Artifact Inspection & Audit Analysis

Directory listings for recovered files were confirmed via tree:

tree carved_usb_output

```
(garba@kali)-[~/usb]
$ tree carved_usb_output
carved_usb_output
├── audit.txt
├── jpg-0-0
│   ├── 00000000.jpg
│   ├── 00000001.jpg
│   ├── 00000002.jpg
│   ├── 00000003.jpg
│   ├── 00000004.jpg
│   ├── 00000005.jpg
│   ├── 00000006.jpg
│   └── 00000007.jpg
├── jpg-1-0
│   ├── 00000008.jpg
│   ├── 00000009.jpg
│   ├── 00000010.jpg
│   ├── 00000011.jpg
│   ├── 00000012.jpg
│   ├── 00000013.jpg
│   ├── 00000014.jpg
│   ├── 00000015.jpg
│   └── 00000016.jpg
└── 3 directories, 18 files
```

Directory tree displaying jpg-0-0 (8 files) and jpg-1-0 (9 files).

Scalpel audit logging was verified using cat:

```
(garba@kali)-[~/usb]
$ cat carved_usb_output/audit.txt

Scalpel version 1.60 audit file
Started at Fri Sep 4 23:57:39 2026
Command line:
scalpel -c /etc/scalpel/scalpel.conf -o carved_usb_output usb_fat_carving.001

Output directory: /home/garba/usb/carved_usb_output
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/garba/usb/usb_fat_carving.001"

The following files were carved:
File           Start           Chop           Length           Extracted From
00000010.jpg    3796992         NO             3148500           usb_fat_carving.001
00000009.jpg    4258222         NO             4875802           usb_fat_carving.001
00000008.jpg    3358722         NO             4965752           usb_fat_carving.001
00000007.jpg    6938624         NO             6868              usb_fat_carving.001
00000006.jpg    5285888         NO             1659604           usb_fat_carving.001
00000005.jpg    3897344         NO             3048148           usb_fat_carving.001
00000004.jpg    15427584        NO             4223822           usb_fat_carving.001
00000003.jpg    12881920        NO             4256310           usb_fat_carving.001
00000002.jpg    19638272        NO             5229050           usb_fat_carving.001
00000001.jpg    17121280        NO             4248530           usb_fat_carving.001
00000000.jpg    21358592        NO             5183267           usb_fat_carving.001
00000016.jpg    36671488        NO             2889262           usb_fat_carving.001
00000015.jpg    36571136        NO             2989614           usb_fat_carving.001
00000014.jpg    36393610        NO             3167140           usb_fat_carving.001
00000013.jpg    36352448        NO             3208302           usb_fat_carving.001
00000012.jpg    35590996        NO             3969754           usb_fat_carving.001
00000011.jpg    35350242        NO             4210508           usb_fat_carving.001

Completed at Fri Sep 4 23:57:40 2026
```

audit.txt file displaying carved image filenames, starting offsets, and byte lengths.

Visual rendering of carved images was confirmed via ImageMagick:

```
display -resize 500x600 carved_usb_output/jpg-0-0/*.jpg
```

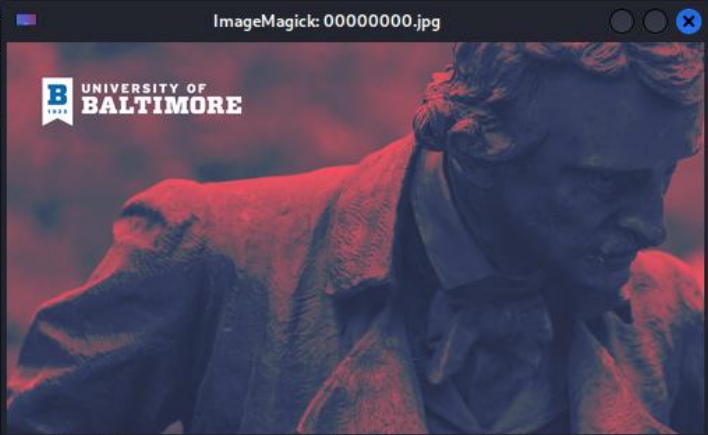
```
scalpel -c /etc/scalpel/scalpel.conf -o carved_
Output directory: /home/garba/usb/carved_usb_ou
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/garba/usb/usb_fat_carving

The following files were carved:
File           Start           Chop
00000010.jpg   3796992          NO
00000009.jpg   425822           NO
00000008.jpg   335872           NO
00000002.jpg   6938624          NO
00000001.jpg   5285888          NO
00000000.jpg   3897344          NO
00000004.jpg   15427584         NO
00000003.jpg   12881920         NO
00000006.jpg   19638272         NO
00000005.jpg   17121280         NO
00000007.jpg   21358592         NO
00000016.jpg   36671488         NO
00000015.jpg   36571136         NO
00000014.jpg   36393610         NO
00000013.jpg   36352448         NO
00000012.jpg   35590996         NO
00000011.jpg   35350242         NO

Completed at Fri Sep  4 23:57:40 2026

(garba@kali)-[~/usb]
$ display -resize 500x600 carved_usb_output/jpg-0-0/*.jpg
```



ImageMagick displaying recovered image artifact 00000000.jpg.

FORENSIC LIMITATIONS & LESSONS LEARNED

1. **Loss of Metadata:** Data carving restores raw image streams from unallocated storage units, but original file names, creation/access timestamps, directory paths, and file system permissions are permanently lost.
2. **Fragmentation Constraints:** File carving tools like Scalpel operate under the assumption that target files occupy contiguous clusters. If a deleted file is fragmented across non-adjacent clusters, recovered outputs suffer structural corruption past the first fragment boundary.
3. **Overwrite Issues:** Storage clusters partially overwritten by subsequent write operations yield corrupted or unrenderable files[cite: 3].